

Deadline Tracker

Team Handbook & Knowledge Guide

Team Nexora

deadline-tracker-team-nexora.vercel.app

November 2026

Team Nexora · Live app: https://deadline-tracker-team-nexora.vercel.app · Code: https://github.com/CodeByDivyansh-tech/DEADLINE-TRACKER-TEAM-NEXORA-

This handbook explains Deadline Tracker so that any member of Team Nexora can explain it, demonstrate it, defend it and discuss its future, even if they built only one part.

Source of truth. Everything here was checked against the project's own code (the same version that is deployed) and its automated tests (269 tests, all passing). Nothing is described as working unless the code does it.

How to read the status labels

Label	Meaning
[Implemented] **Implemented**	Works today in the live app
[Partial] **Partial**	Works, but with a clear limit (explained next to it)
[Planned] **Planned**	An idea for the future; not in the app yet

Contents

1. Project introduction
2. Problem statement and solution
3. Complete feature guide
4. Why Deadline Tracker is different
5. User guide: how to use the app
6. Deadline AI: complete explanation
7. Technical architecture
8. Security and privacy
9. Real-life use cases
10. FAQ (70 questions)
11. Difficult questions from judges and faculty (32 questions)
12. Demonstration scripts (5 minutes and 2 minutes)
13. Team preparation
14. Future roadmap
15. Glossary

1. Project introduction

What is Deadline Tracker?

Deadline Tracker is a web app for managing tasks and deadlines. A normal to-do list remembers what you have to do. Deadline Tracker also checks **whether you will finish on time**: it compares how much work is left with how much time you have, warns you early with a reason, and helps you plan the work backwards from the due date.

It works for three kinds of users, chosen when you start:

- **Student**: assignments, exams, projects, subjects, a study planner.
- **Company**: team deadlines, workload, approvals, a "Needs attention" list for managers.
- **Personal**: bills, appointments, life admin.

It has an assistant called **Deadline AI** with two separate parts:

- **My Plan**: plans your own tasks ("plan my tomorrow"). It runs on our own server with clear rules and never sends your tasks to an outside AI.
- **Help**: answers questions, using Google Gemini (free tier). Website questions are answered only from our help guide.

Why did we build it?

Most students don't miss deadlines because they forgot them completely. They miss them because they **started too late**, had **several deadlines in the same week**, or didn't notice that a big task needed more hours than they had left. A list that only shows due dates can't warn about that. We wanted an app that warns early and helps with the next step.

Who needs it?

- Students with several subjects, assignments, labs and exams at the same time.
- Project teams and small teams that need to see who is overloaded.
- Anyone juggling college, internship and personal work.

Why aren't ordinary to-do lists enough?

A typical to-do list…	What students actually need…
Shows the due date	To know if there's still **enough time** to finish
Goes red when it's too late	A warning **before** it's too late, with the reason
Treats every task as one tick-box	To split big work into daily pieces
Doesn't know about your other deadlines	To see when several deadlines **clash**
Needs you to fill forms	To type naturally: "DBMS assignment due Fri 5pm"

Our solution in one sentence

A to-do list remembers your tasks; Deadline Tracker warns you before you fall behind, tells you why, and plans the work for you to approve.

Main objective

Help people finish work on time with less stress, through early warnings, simple planning and reminders, while keeping their data private.

Introductions you can memorise

One line

> Deadline Tracker is a task app that tells you whether you'll finish on time, not just what is due.

30 seconds

> Students rarely forget deadlines completely; they start too late or have three deadlines in one week. Deadline Tracker lets you type tasks the way you'd say them, like "Java assignment due Friday 5pm". It then checks how much work is left against the time you have and warns you early, with the reason. Its assistant can plan your day from your own tasks, and reminders arrive even when the app is closed.

One minute

> Deadline Tracker is a web app for students, teams and personal use. You add work in one box in plain English, or even Hinglish. A risk radar marks each deadline On track, At risk, Critical or Overdue, and shows why, for example "due in 2 days, 40% done, 6 hours left". It can plan the work backwards into daily pieces, warn when deadlines clash, and suggest a rescue plan when something is in trouble. Deadline AI has two parts: My Plan, which plans your day from your own tasks on our server, and Help, which answers questions with Google Gemini using our help guide. Accounts keep each person's data private. The server sends reminders at the right time in your time zone, even while the app is closed. It runs live on Vercel and Render.

Three minutes (use the one-minute version, then add these points)

> 1. *The problem:* deadlines are missed because of late starts and clashes, not only forgetting.
> 2. *How it works:* each task can have a day to work on it and a hard deadline. The radar compares work left (your estimate minus progress) with your available hours (set in Settings), so it's a clear rule you can explain, not a black box.
> 3. *Planning:* backward plans, a study planner, a calendar you can drag tasks on, a "plan my tomorrow" command, and Rescue mode for overdue work, which can even draft an extension request you send yourself.
> 4. *Privacy by design:* your tasks are never sent to an outside AI. Gemini only sees your Help question and public help text. Passwords are stored as secure hashes, and every request is checked against the logged-in account.
> 5. *Engineering:* a plain JavaScript frontend, a Node.js backend with a SQLite database, a reminder job that runs every minute, and 269 automated tests. The frontend is on Vercel and the backend on Render.
> 6. *What's next:* email or Telegram reminders, shared team workspaces and calendar sync.

2. Problem statement and solution

The problems

1. **Forgetting deadlines**, especially ones announced once in class or on a group chat.
2. **Too many tasks at once**: several subjects, labs, a project and an exam in the same week.
3. **Poor prioritisation**: working on the easy task instead of the urgent one.
4. **Last-minute work**: starting a 10-hour assignment the night before.
5. **No daily plan**: knowing what's due, but not what to do *today*.
6. **Losing track** of what's pending and what's done.
7. **Clashing deadlines** that only become obvious when it's too late.

How Deadline Tracker addresses each one (only what the app really does)

Problem	What the app does	Status
Forgetting deadlines	Add in seconds from one box; reminders 1 day and 1 hour before (configurable); a morning reminder on the due day; overdue alerts. With an account, the server creates reminders even while the app is closed.	[Implemented] (in-app bell + desktop alerts while open; no email/SMS yet)
Too many tasks	Workspaces, subjects/projects, filters, search, Today / Upcoming / Done tabs	[Implemented]
Poor prioritisation	Risk radar with reasons; "I have too much work" in My Plan lists tasks in order of urgency with reasons	[Implemented]
Last-minute work	Estimates + backward plan into daily pieces; study planner spreads sessions; radar warns when the hours left don't fit	[Implemented] (needs an estimate to be accurate)
No daily plan	Calm Today page; "plan my tomorrow" builds a plan from saved tasks for you to approve	[Implemented]
Pending vs done	Done tab, progress check-ins (0–25–50–75–100%), Reports and account statistics	[Implemented]
Clashing deadlines	Clash detection and "Overloaded this week"; the workload view for teams	[Implemented]

Honest limits: the app can only warn about work it knows about. If a task isn't entered, it can't help. Risk accuracy depends on the estimates you give. It doesn't read your college portal or WhatsApp groups automatically.

Student-life examples

- **Riya** has a DBMS assignment (Friday 5 pm, about 5 hours), a Java lab record (Thursday) and a Maths test (Monday). She types: `DBMS assignment due Fri 5pm ~5h, Java lab record due Thu, Maths test Monday`. The app creates three deadlines. With 3 study hours a day it plans DBMS work on Tue–Thu, and if the hours don't fit, the radar shows **At risk** with the reason.

- **Arjun** forgot a physics lab report that was due yesterday. It shows **Overdue**; **Rescue mode** breaks the rest into small steps, makes a revised schedule and drafts a polite message to his teacher asking for an extension. He copies the message and sends it himself; the app sends nothing.
- **A final-year project team** in a Company workspace sees who has too many hours this week on the Workload page, and moves a task.

3. Complete feature guide

Each feature has: what it is, why it exists, how to use it, what happens inside, benefits and limits.

3.1 Accounts and log-in [Implemented]

- **What:** create an account (name, email, password of 8+ characters), log in and log out. The first account on a server becomes the **admin**.
- **Why:** so your tasks are saved safely on the server and nobody else can see them.
- **How:** open the site, choose **Create account** or **Log in**. To log out, use **Log out** at the bottom of the sidebar, or Settings → Profile.
- **Inside:** the password is turned into a secure hash (scrypt with a random salt) and only the hash is stored. The server sets a session cookie the page's scripts can't read (HttpOnly), valid for 30 days. Only a fingerprint (SHA-256) of the session token is stored.
- **Benefits:** the same tasks on any device after logging in; privacy between users.
- **Limits:** there's no "forgot password" email yet, because no email service is connected. A password can be changed while logged in, through the server's profile API, but there is no button for it in the app yet. [Partial]

3.2 First-time setup and workspaces [Implemented]

- **What:** a 3-step setup: your name → Student, Company or Personal → optional sample data. You can create more workspaces later.
- **Why:** students and teams need different words and pages, such as "Subject" versus "Project".
- **How:** Settings → Profile → **New workspace**; switch workspaces from the box at the top of the sidebar.
- **Limits:** a workspace belongs to one account. Team members in a Company workspace are names you add, not real accounts yet (see 3.20). [Partial]

3.3 Sample data [Implemented]

New workspaces can start with example deadlines so you can try everything. A **Sample data** badge shows at the top; **Clear** removes them.

3.4 Today page (the dashboard) [Implemented]

- **What:** the home page. It shows a greeting with the date, the **Add box**, one line for "things that need your attention", **today's task list** with Today / Upcoming / Done tabs and a progress ring, and **Next deadlines**.
- **Why:** to answer "what should I do now?" without a long, noisy page.
- **Progress check-in:** sometimes the app asks "How far along is *X*?" with 0% / 25% / 50% / 75% / Done buttons. Your answer updates the progress, which makes the risk level more accurate. "Not now" skips it.

3.5 Adding tasks: one Add box [Implemented]

- **What:** one box that understands normal sentences. It's on the Today page; press **N** anywhere for **Quick add**.
- **How the text is read:**
  - **Deadline vs to-do:** "due", "deadline", "submit by", or an exam, test, quiz or viva makes a **deadline**. Anything else is a **to-do** planned for a day. No date means today.
  - **Several at once:** separate tasks with commas or new lines. A date said once at the start applies to the whole list.
  - **Dates:** today, tomorrow, day after tomorrow, Friday, next Monday, end of week, this weekend, 5 Nov, 2026-11-05.
  - **Times:** 5pm, 17:30, noon, morning, evening.
  - **Effort, priority, repeats:** effort `~3h` or `30 min`; priority `!high`, `urgent`, `!low`; repeats `every Monday`, `daily`.
  - **Subject and person:** subject or project `#dbms`; a person `@rahul` or "assign the slides to Rahul" (managers).
  - **Spelling:** common misspellings like "tommorow" are understood.
- **Inside:** a rule-based parser (`parseQuick`) turns the sentence into fields; nothing is guessed. If no date is mentioned in the assistant, no date is set.
- **Example:** `DBMS assignment due Fri 5pm ~5h, buy notes, call Priya tomorrow` → 1 deadline + 2 to-dos.
- **Limits:** it's rule-based, so unusual sentences may need a quick edit afterwards.

3.6 Editing, completing, moving and deleting [Implemented]

- **Complete:** tick the round box. It moves to Done; untick to reopen.
- **Edit:** the pencil icon (or click the task) opens the **task panel**: title, day, time, deadline, priority, status (To do, In progress, Blocked, In review, Done), estimate, progress, notes, reminders, steps (milestones), what it depends on, and a link.
- **Move to tomorrow:** the arrow icon. It changes only the planned day, never the deadline.
- **Delete:** the bin icon. An **Undo** button appears for a few seconds.
- **Undo:** most changes (delete, move, edits from the assistant, extensions) show an Undo in the message at the bottom of the screen.

3.7 Deadlines page and the Deadline Radar [Implemented]

- **What:** all deadlines with filters (type, subject, risk, status, person), grouping, search, and the **Deadline Radar**, a picture where each dot is a deadline placed by urgency.
- **Risk levels:** On track · Needs attention · At risk · Critical · Overdue. Each shows its **reason**, such as "due in 2 days, 40% done, about 6h left".
- **Inside:** a plain rule, not AI. Work left = estimate × (1 − progress), including deadlines due earlier. This is compared with the hours you have before the due date (Settings → Availability). The thresholds can be changed in Settings → Availability → Advanced. Tasks it depends on that aren't finished raise the level.
- **Limits:** without an estimate, the radar can only use the date. A "Needs an estimate" list reminds you to add one.

3.8 Planning tools [Implemented]

- **Backward plan:** in a deadline's panel, **Suggest a backward plan** splits the work into dated steps before the due date. You edit them and accept.
- **Study Planner** (students): spreads study sessions for your deadlines and exams over the coming days, within your hours per day, and lists what **won't fit in time**.
- **Clash detection:** warns about overloaded weeks and deadlines piling up.
- **Rescue mode:** for deadlines that are Critical or Overdue (or At risk), it walks you through four steps: (1) break the rest into small steps; (2) make a revised schedule up to a target day; (3) clear blockers; (4) **ask for an extension** with a ready-made message you copy and send yourself. If the extension is granted, you record it and the deadline moves only then, with Undo. The app never sends the message and never moves a deadline by itself.
- **Focus sessions:** a 25-minute focus timer from a task. The time logged feeds **Estimate accuracy** in Reports, which compares your estimates with the real time spent.

3.9 Calendar [Implemented]

Month and week views of tasks and deadlines. **Drag a task to another day** to plan it for that day. Today, previous and next buttons are included.

3.10 Exams page (students) [Implemented]

Upcoming and past exams with preparation progress; topic checklists you can tick as "revised".

3.11 Search and filters [Implemented]

The search box at the top finds tasks by title. The Deadlines page has filters and grouping. The server's task API also supports search and filters (see section 7).

3.12 Reminders and notifications [Implemented / Partial]

- **What you get:** reminders before deadlines (default 1 day and 1 hour; choose others), a reminder 15 minutes before timed to-dos, a morning reminder on the due day, overdue alerts, a daily summary ("You have 3 pending tasks today"), and "You've completed all your high-priority tasks for today".
- **Where:** the **bell** at the top (with an unread count), plus **desktop alerts** while the app is open, if you turn them on. **Quiet hours** stop pop-ups at night.
- **Inside:** with an account, a job on the server runs every minute. It works out the reminders in **your time zone**, stores each one once (never duplicated), and marks it delivered on time, **even while the app is closed**. When you open the app, it's waiting in the bell.
- **Limits:** [Partial] no email, SMS, WhatsApp or phone push yet. On Render's free plan the server sleeps after 15 minutes without visits, and reminders are created when it wakes.

3.13 Deadline AI: floating window and Assistant page [Implemented]

See section 6 for the full explanation. In short: the round **Deadline AI** button (bottom right) opens a window with two tabs:

- **My Plan:** your tasks, planned by our server's engine.
- **Help:** questions answered with Gemini, using our help guide for website questions.

3.14 Reports and statistics [Implemented]

- **Your account** card (from the server): total, completed, pending, overdue, completion %, on-time %, tasks completed per week (8 weeks), monthly totals, and pending by priority.
- **Workspace reports:** completion rate, on-time rate, average finish, completed per week (on time vs late), current risk mix, estimate accuracy, and progress by subject or project.

3.15 Settings [Implemented]

Settings is split into short sections:

- **Profile** (name, workspace, account, log out)
- **Availability** (study or work hours per day, work days, risk rules)
- **Reminders** (times, overdue alerts, daily summary, due-day reminder, high-priority done, quiet hours, desktop alerts)
- **Assistant** (what My Plan and Help use)
- **Subjects / Projects / Lists** (names and colours)
- **Team** (Company only)
- **Data** (export to calendar, backup and restore)

3.16 Light and dark theme [Implemented]

The moon/sun button at the top switches the theme. Both themes are checked for readable contrast by automated tests.

3.17 Export and backup [Implemented]

- **Export to calendar (.ics):** download your deadlines for Google Calendar, Outlook or Apple Calendar. This is a one-time file, not live sync.
- **Copy backup / Restore:** copy all your data as text and paste it back later.

3.18 Phone layout [Implemented]

The same website adapts to phones: a bottom bar for the main pages, a full-screen Deadline AI window, and no sideways scrolling (tested at 390 px wide). It isn't an installable app yet. [Planned]

3.19 Admin dashboard (`/admin`) [Implemented]

For admins only (checked on the server):

- **Overview:** users, tasks, knowledge, unanswered questions, ratings, AI calls, errors.
- **Users:** change roles, turn accounts off or on, delete. The last admin is protected. Admins **can't see anyone's tasks**.
- **Knowledge:** add, edit and delete help articles and FAQs; upload .md, .txt or .html files; refresh the search index.
- **Questions:** questions Help couldn't answer (turn one into an FAQ in one click), common questions, rated answers.
- **AI instructions:** tone rules for Help; switch **Any question** / **Focused**.
- **API usage:** Gemini calls per day, quota hits, tokens. **Errors:** recent server errors.

3.20 Company features [Partial]

- **What works:** My Work, Team Deadlines, **Workload** (each person's hours versus capacity, with suggestions to move work), **Needs attention** (overdue, at risk, blocked, waiting for approval), submit for review → approve or request changes, roles (Owner, Admin, Manager, Employee, Guest).
- **Limit:** team members are names inside one account's workspace; they don't log in separately yet. The "Viewing as" switch shows how each role would see the workspace, and the app says plainly that permissions there are simulated. Real shared workspaces with invites are planned. [Planned]

3.21 Works without the server too [Implemented]

If the page is opened somewhere without our backend, it still works and saves in that browser, with simple planning rules. On the live site you always use the server and your account.

3.22 Not in the app

- **Reader mode:** discussed earlier and deliberately skipped.
- **Live Google Calendar sync, email or SMS reminders, a native mobile app, real team invites:** planned (section 14).

4. Why Deadline Tracker is different

What existing tools generally offer

These are general descriptions; check each product's current features before quoting them.

- **Google Calendar:** events and time blocks, reminders, simple tasks, sharing. It's excellent for *when* things happen.
- **Microsoft To Do:** lists, due dates, reminders, a "My Day" view, syncing across devices.
- **Notion:** very flexible pages and databases, templates, team collaboration, with AI features available as an add-on. You build your own system.
- **Todoist:** fast task entry with natural-language dates, priorities, labels, filters, projects, reminders.
- **Paper planners:** simple and distraction-free, but no reminders, search or recalculation.
- **Basic to-do apps:** add, tick, delete.

Where our approach is different

Question	Typical to-do / calendar app	Deadline Tracker (today)
Will I finish on time?	Usually not answered; shows dates	[Implemented] Risk level from work left vs hours available, **with the reason**
Plan backwards from a deadline	Manual	[Implemented] Suggested steps and study sessions before the due date
Clashing deadlines	You notice yourself	[Implemented] Overloaded weeks and clash warnings
Already behind?	Item turns red	[Implemented] Rescue mode: smaller steps, revised schedule, extension message
Plan my day	Manual, or AI features in some tools	[Implemented] "plan my tomorrow" from your own tasks, with a reason for each, saved only after you approve
Typing tasks	Forms; some tools understand dates	[Implemented] One box understands lists, deadlines, effort, priority, Hinglish and typos
Private data and AI	Varies by tool	[Implemented] Tasks never go to an outside AI; Gemini sees only Help questions and public help text
Student, team and personal modes	Usually one style	[Implemented] Three workspace types with matching words and pages
Learning your estimates	Rare	[Implemented] Estimate accuracy from focus sessions

Be fair when presenting: Google Calendar, Todoist and Notion are mature, polished products with features we don't have: live sync, native apps, integrations, large teams. Some of them also offer AI features or understand natural-language dates. Don't claim they "can't" do something. Our

difference is the **focus**: we answer "am I going to make it, and what should I do now?", and we're open about how the answer is calculated.

Why a student would choose it

1. It warns *before* trouble, with a reason they can understand.
2. Adding work is as fast as typing a message, including Hinglish.
3. It helps with the next step (a plan, a rescue plan), not just a list.
4. One app for college, internship and personal life.
5. Privacy: tasks aren't fed to an AI.

How the AI direction helps

- Deadline AI turns messy sentences into tasks and plans the day, but always as **suggestions you confirm**.
- Help answers how-to questions instantly and admits when it doesn't know, which builds trust.
- Keeping planning on our own rules means it works even when the AI quota runs out.

Possible future advantages [Planned] (not built yet)

Live calendar sync, college portal or timetable import, email, Telegram or WhatsApp reminders, shared team workspaces, an installable phone app, and smarter estimates learned from your history.

5. User guide: how to use the app

5.1 Open and sign in

1. Open **https://deadline-tracker-team-nexora.vercel.app**.
2. If you see **"Starting up..."**, wait up to a minute: the free server is waking up.
3. Click **Create account**, enter your name, email and a password (8+ characters). Next time, use **Log in**.
4. Setup: type your first name → choose **Student** → leave **Start with sample deadlines** on to explore, or off to start empty → **Done**.

5.2 Find your way around

- **Computer:** the sidebar has **Today, Assistant, Calendar, Deadlines** and **More** (Exams, Study Planner, Reports, Settings). The bell and the theme button are at the top right.
- **Phone:** the same pages are in the bottom bar.
- **Deadline AI:** the round button at the bottom right of every page.

5.3 Understand the Today page

- **Greeting and Add box**: where you type new work.
- **"N things need your attention"**: click to see what's risky.
- **Today list**: tabs for **Today** (planned or due today, plus unfinished work carried over), **Upcoming** and **Done**.
- **Next deadlines**: the closest deadlines, coloured by risk, sometimes with a progress check-in.

5.4 Add a task or deadline

1. Click the Add box (or press **N** anywhere).
2. Type naturally, for example:
    - `Java assignment due Friday 5pm` → a **deadline** on Friday at 5:00 pm
    - `Maths test next Monday 10am` → a **deadline** (a test counts as one)
    - `Practical file submission due 5 Nov ~3h !high` → a deadline with a 3-hour estimate and high priority
    - `Revise unit 2 tomorrow` → a **to-do** planned for tomorrow
    - `College project report due Sat, buy chart paper, call teammate tomorrow` → three items at once
3. Press **Add** (or Enter).

5.5 Set or change the deadline and priority

1. Click the task (or its pencil icon) to open its panel.
2. Change the **Deadline** date and time, the **Priority** (High / Medium / Low), the **Estimate** in hours, the **Status** and the **Progress**.
3. Changes save by themselves; use **Undo** if needed.

Tip: always add an **estimate**. The radar and planners become much more accurate.

5.6 Mark complete, move, delete

- **Complete:** tick the circle next to the task.
- **Move to tomorrow:** the arrow icon. The deadline stays the same.
- **Delete:** the bin icon, then **Undo** if it was a mistake.

5.7 Find pending tasks

- Today page → **Upcoming** tab for what's next; **Done** for finished work.
- **Deadlines** page → filter by status, risk, subject or type.
- Ask Deadline AI → My Plan: `what's due this week?` or `what's overdue?`

5.8 Search

Type in the **Search** box at the top (on a computer) to find tasks by title.

5.9 Use the calendar

1. Open **Calendar** and choose **Month** or **Week**.
2. **Drag** a task to another day to plan it for that day.

5.10 Plan backwards and use Rescue mode

- Open a deadline → **Suggest a backward plan** → edit the steps → accept.
- For a Critical or Overdue deadline → **Rescue** → follow the four steps. Copy the extension message and send it yourself; if your teacher agrees, record **Yes** and the deadline moves (with Undo).

5.11 Reminders

1. **Settings → Reminders**: choose times (e.g. 3 days, 1 day, 1 hour before), overdue alerts, daily summary, due-day reminder, the high-priority-done message, and quiet hours.
2. Click **Turn on desktop alerts** and allow notifications in your browser to get pop-ups while the app is open.
3. Check the **bell** for everything that arrived while you were away.

5.12 Use Deadline AI

1. Click the **Deadline AI** button.
2. **My Plan** tab (your own tasks):
    - `I have a Java assignment tomorrow and a Maths test on Friday.` → suggested tasks → edit or untick → **Save to my tasks**
    - `plan my tomorrow` → a plan from your saved tasks, most urgent first, with a reason for each → **Save**
    - `I have too much work. Help me organise it.` → an order of urgency, overloaded days, what won't fit

    - `break DBMS assignment into smaller steps` → work sessions before the deadline
    - Hinglish works: `kal Java assignment submit karna hai`, `aaj kya karu`
3. **Help** tab (questions):
    - `How do I delete a task?` → answered from the help guide, with the source shown
    - `How can I stop procrastinating?` or `Explain recursion in Java` → a general answer from Gemini
    - Use thumbs up / thumbs down to rate answers. **Clear** starts a new conversation.

5.13 Change the theme

Click the **moon / sun** button at the top right.

5.14 Reports, export and backup

- **More → Reports** for your statistics.
- **Settings → Data → Export to calendar (.ics)**, **Copy backup**, **Restore from backup**.

5.15 Teams (Company workspace)

Settings → Profile → **New workspace** → Company. Add people in Settings → Team. Assign work with the **For** menu next to the Add box or by typing `@name`. Use **Workload** and **Needs attention** to manage the team.

6. Deadline AI: complete explanation

What it is

Deadline AI is the app's assistant. It has **two deliberately separate parts**:

	My Plan	Help
Purpose	Plan *your* tasks	Answer questions
Data it uses	Your saved tasks (on our server)	Your question + public help guide
How it thinks	Our rule-based planning engine (no outside AI)	Google Gemini (free tier), with RAG for website questions
Can it change your tasks?	Only after you click **Save**; the server checks everything	Never
Status	[Implemented]	[Implemented] when the server has a `GEMINI_API_KEY`; otherwise it answers from the guide only

Why split it?

Google may use free-tier Gemini requests to improve its products. Sending students' private schedules there would be a privacy risk, so **private planning stays on our server** and only public, non-private text goes to Gemini.

What My Plan can do [Implemented]

- Turn sentences (English or Hinglish, with typos) into suggested tasks and deadlines.
- Plan a day: overdue work first, then deadlines due that day or the next, then work set aside for bigger deadlines, then left-over to-dos. It warns if the day is over your hours.
- Organise everything ("too much work"): an order of urgency with reasons, overloaded days, tasks that won't fit, tasks missing estimates.
- Break a task into work sessions before its deadline. **It asks for an estimate instead of guessing.**
- Move unfinished tasks, move one task, mark one done, list what's overdue or due, give your statistics.
- Spot duplicates (unticked automatically) and suggest an optional revision session before a test (unticked by default).

How My Plan works, step by step

1. You type a message; the app sends it to `/api/plan/chat`.
2. The server decides what you're asking (plan a day, organise, break down, add tasks...) using clear patterns.
3. The planning engine reads **only your** tasks from the database and applies its rules.

4. It returns a reply and a list of **suggested actions**, stored with the conversation.
5. You can edit or untick suggestions. Only **Save to my tasks** calls `/api/plan/apply`.
6. The server reloads the stored suggestion, allows only safe edits (keep, title, date, time, priority), checks that every task belongs to you, and writes to the database in one transaction.
7. It **never moves a hard deadline** unless you explicitly asked for that, and the change is shown first.

How Help works (RAG), step by step

1. You ask a question in the Help tab; the app sends it to `/api/chat`.
2. **Retrieval:** the server searches our help guide in two ways at once:
    - **By meaning:** your question is turned into a list of numbers (an *embedding*) by Gemini's embedding model, then compared with the stored embeddings of every guide section (cosine similarity).
    - **By keywords:** a classic search (BM25) that understands Hinglish words like *kaise*, *hatana* and *kaam*, and fixes spelling like "delte a taks".
    - The two rankings are combined, and the best 4 sections are kept only if they're good matches.
3. **Generation:** Gemini receives our instructions, the recent Help conversation, your question and those 4 sections. It must reply in a fixed JSON format saying whether it found an answer, what kind (website or general) and which sections it used.
4. **Checks:** an answer about the website that isn't backed by the guide is **blocked**, and you see "I don't know"; the question is then listed for the admin. Unreadable answers are never shown.
5. **Answer:** you see the answer, its source sections and a label (e.g. "Google Gemini · from the help guide"). You can rate it.

Key terms in simple words

- **RAG (retrieval-augmented generation):** first *find* the right information, then let the AI *write* an answer using only that. It stops the AI inventing website features.
- **Embedding:** a list of numbers that represents the meaning of a text; texts with similar meaning have similar numbers.
- **Vector database:** a store of embeddings that can find the closest ones. **We don't use a separate vector database product**: the embeddings are stored in our SQLite database and compared in our server code, which is enough for a help guide of this size.
- **Gemini API:** Google's AI service. We call it only from the server. The models are `gemini-3.5-flash-lite` (answers) and `gemini-embedding-001` (embeddings), both listed by Google as free on the free tier when we set this up.

What Help answers

- **Any question** mode (default): website questions (only from the guide), study and productivity tips, and general questions (coding, science, writing...). It says when it's unsure, and refuses harmful requests.
- **Focused** mode (admin switch): only the website and productivity tips, which saves the free quota.

What happens at the free-tier limit

- Our server has its **own limits** below Google's: 8 answers per minute, 400 per day for the whole site, 40 per person per day (all configurable).
- If Google replies "quota exceeded" (HTTP 429), the server pauses AI calls for the time Google asks.
- In all these cases Help still answers **website questions straight from the guide** and says honestly that the AI limit was reached. There is **no paid fallback**.
- **My Plan is unaffected**, because it doesn't use Gemini.

Three different kinds of "intelligence" in the app

Kind	Examples	Who decides	Can change data?
AI-generated answers	Help replies	Gemini, limited by our rules and the guide	No
Rule-based calculations	Risk radar, backward plans, day plans, workload, reminders	Our code, with explainable rules	No (only suggests)
Database operations	Saving tasks, applying a confirmed plan	Our server, after login and ownership checks	Yes, only after your action

How it's different from a normal chatbot

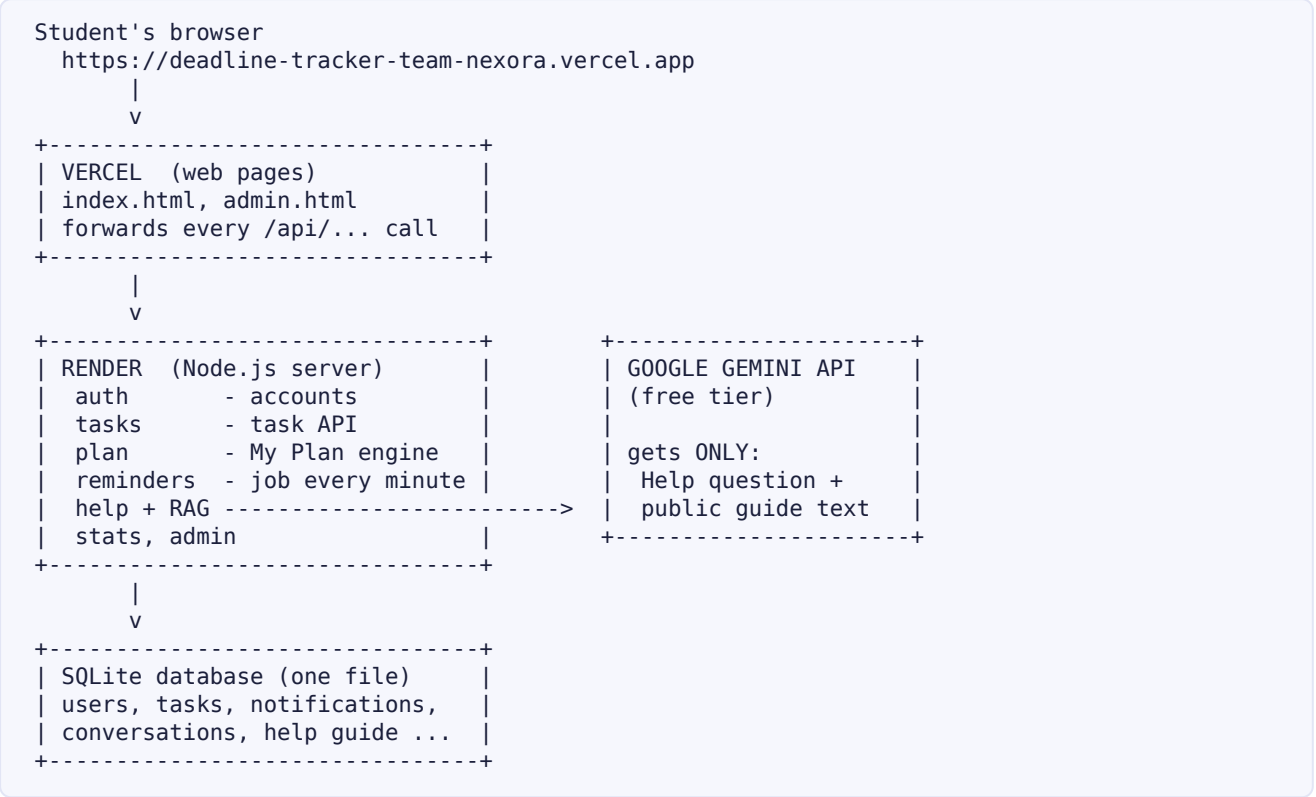
- It knows the app's real features (RAG) and says "I don't know" instead of guessing.
- Planning uses your real tasks and returns **actions you approve**, not just text.
- Private planning never touches an outside AI.

7. Technical architecture

Technology actually used

Part	What we use
Frontend	One HTML page with **plain JavaScript** (no framework such as React) and **hand-written CSS** (no CSS framework). The page is built from parts in `src/` by `build.js`.
Backend	**Node.js 22.5+** using only built-in modules (`http` , `crypto` , `node:sqlite`). No runtime npm packages.
Database	**SQLite** (one file) with numbered migrations
Authentication	Our own: scrypt password hashes, session cookie (HttpOnly, SameSite=Lax, Secure in production), session tokens stored as SHA-256
AI	Google **Gemini API** (free tier): `gemini-3.5-flash-lite` + `gemini-embedding-001`
Hosting	**Vercel** (frontend pages) + **Render** (backend server, free plan)
Code hosting	GitHub
Testing	Node's built-in test runner + **Playwright** (real Chromium browser): 269 tests

Architecture diagram



Because Vercel forwards `/api` to Render, the browser talks to only **one address**. That keeps log-in cookies simple and safe (no cross-site cookies).

How a request travels (example: adding a task)

1. You type `Java assignment due Fri 5pm` and press Add.
2. The page reads it into a task and saves its state: `PUT /api/state` (with the version it last saw).
3. Vercel forwards the request to Render.
4. The server checks the session cookie → finds *your* account → checks the request came from our site.
5. It validates every task (dates, lengths, values) and writes tasks to the `tasks` table (only rows with your user id) and the rest (workspaces, settings) to `app_state`, in one transaction.
6. It replies with the new version number. If another device saved first, it replies **409 Conflict** with the newest copy, so nothing is silently overwritten.
7. The reminder job is nudged to reschedule your reminders.

How an AI request travels (Help)

Question → `/api/chat` → login check → search the guide (embeddings + keywords) → Gemini with the top sections → JSON answer → checks (no unsupported website claims) → saved in your conversation and in the questions log → shown with sources.

Database tables (simplified)

Table	Holds
`users`	name, email, password hash + salt, role (user/admin), time zone, reminder preferences
`sessions`	hashed session tokens, expiry
`tasks`	title, description, due date/time + UTC moment, planned day/time, priority, status, category, estimate, progress, reminders, extra details (steps, links…)
`app_state`	workspaces, settings and chat (everything except tasks), with a version number
`notifications`	message, scheduled time, delivery status, read time, unique key per user
`conversations`, `messages`	Deadline AI history (Help and My Plan)
`kb_sources`, `kb_chunks`	help guide articles, their sections and embeddings
`questions`	Help questions for the admin (answered or not, ratings)
`ai_usage`, `error_log`	Gemini calls and server errors, for the admin

Constraints (e.g. priority must be high/medium/low) and indexes (by user, status and due time) are defined in the database itself.

Main API groups

`/api/auth/*` (sign up, log in, log out) · `/api/profile` · `/api/tasks` (create, list with filters, today/tomorrow/overdue/upcoming, update, complete, reopen, delete) · `/api/state` (the app's save)

· `/api/plan/*` (My Plan) · `/api/chat` (Help) · `/api/notifications` · `/api/stats` · `/api/conversations` · `/api/admin/*` · `/api/health`. Full list: `docs/API.md`.

Reminder job

Every minute (and right after a task changes) the server works out each user's reminders in their own time zone, stores each with a unique key (so it can't be sent twice), cancels reminders for finished or moved tasks, and marks due ones as delivered. If the server was off, only the latest reminder for a task is delivered, so there's no pile-up.

8. Security and privacy

What's protected, and how [Implemented]

- **Passwords:** never stored as text. Each one is hashed with **scrypt** (a slow, memory-hard algorithm) and its own random salt. Log-in errors say "Email or password is wrong" without revealing whether the email exists.
- **Sessions:** a random 256-bit token in a cookie that page scripts can't read (**HttpOnly**), not sent from other sites (**SameSite=Lax**), and HTTPS-only in production (**Secure**). Only the token's SHA-256 is stored, so a stolen database copy can't be used to log in. Log-out deletes the session. Changing the password signs out other devices.
- **Separation between users:** every database query for private data includes the logged-in user's id taken from the session, **never** an id sent by the browser. Asking for someone else's task returns "not found". The tests check this for tasks, plans, notifications, conversations and ratings.
- **Admins** can manage accounts and help content but **can't read anyone's tasks**.
- **Input checks:** every field is validated (lengths, dates, allowed values). The database also has its own constraints. All SQL uses parameters, which prevents SQL injection; a test searches for `' OR 1=1 --` and gets no results.
- **Web protections:** a Content-Security-Policy, X-Frame-Options DENY (no embedding in other sites), nosniff, and a same-origin check that blocks state-changing requests from other websites. Request bodies must be JSON and are size-limited.
- **Rate limits:** 10 wrong log-ins per 15 minutes per email and address, 20 sign-ups per 15 minutes, 600 API requests per minute per address, plus AI limits per minute, per day and per user.
- **Keys and secrets:** the Gemini key lives only in Render's environment settings (or a local `.env` file that's in `.gitignore`). It's never in the frontend, never in GitHub, and never in any response (tested).
- **Admin routes** are checked on the server: all 15 refuse normal users (403) and visitors (401), which is tested.

What is sent to outside services

- **Google Gemini:** only the **Help** question, the recent Help conversation and matching public help-guide text. **Never tasks, deadlines, schedules or account details.** The Help window says that Google may use free-tier requests to improve its products and asks users not to type private details.
- **Vercel and Render:** they host the app, so data passes through them, encrypted with HTTPS.

Known limits and needed improvements [Partial]

- **Render free plan:** the database file is erased on every redeploy or restart, and the server sleeps when idle. Real use needs a paid plan with a persistent disk, or a managed database. This is a data-durability issue, not a security hole, but users must know it.
- No email verification or password-reset email yet (no email service).

- No two-factor authentication.
- The admin dashboard has no audit log of admin actions.
- Rate limits are kept in server memory (fine for one server; several servers would need a shared store).
- Data at rest isn't encrypted beyond what the host provides.
- The Content-Security-Policy allows inline scripts, because the app is one self-contained page. Moving scripts to separate files would allow a stricter policy.

9. Real-life use cases

1. **First-year engineering student.** Types the week's work from the class group in one go: `Physics lab record due Thu, Maths tutorial due Fri, buy lab coat tomorrow`. Sets study hours (3/day) in Settings. Checks Today each morning; the bell reminds the day before.
2. **Student with several assignments.** Adds estimates (`~5h`, `~2h`). The radar shows DBMS **At risk** because it needs 5 hours by Friday but only 4 are free after other work. The backward plan spreads sessions from Tuesday.
3. **Exam preparation.** Adds `DBMS exam 15 Nov`. Uses the Exams page topic checklist to tick revised units and the Study Planner to spread revision. In My Plan, `break DBMS revision into smaller steps` (after giving an estimate) creates dated sessions.
4. **Laboratory practicals.** Adds `practical file submission due 5 Nov`, and `write experiment 4 every Monday` as a repeating to-do. Uses steps (milestones) for each experiment write-up.
5. **Final-year project team.** A Company workspace: the leader adds members, assigns `@rahul slides`, `@sneha report chapter 3`. **Workload** shows who is overloaded; **Needs attention** shows blocked and late items; members submit work for review and the leader approves. *(Members are names in the leader's workspace; separate logins are planned.)*
6. **Internship applicant.** A Personal or Student workspace with `Google internship application due 20 Nov`, `update resume tomorrow ~2h`, `mock interview Sat 6pm`. Reminders 3 days and 1 day before.
7. **College club event.** A Company workspace for the event: `book auditorium due Mon`, `posters due Wed`, `sponsor emails @aman`. The calendar shows the week; drag tasks to rebalance.
8. **Freelancer.** A Company or Personal workspace per client: deliverables with deadlines and estimates; the radar warns when two clients' deadlines clash; Reports show on-time rates.
9. **Working professional.** A Company workspace: My Work for their own tasks, the morning daily summary, quiet hours at night, and "plan my day" in My Plan.
10. **Personal goals.** A Personal workspace: `pay electricity bill 5 Nov`, `gym every day 7am`, `passport renewal deadline 20 Nov`. Overdue alerts make sure bills aren't missed.
11. **A student who fell behind.** An overdue lab report → **Rescue mode** → a revised schedule plus an extension message to the teacher → records the teacher's answer; the deadline moves only if approved.

10. FAQ (70 questions)

Short answers first; longer ones where the question is technical.

A. General project questions

1. **What is Deadline Tracker?** A task and deadline app that tells you whether you'll finish on time and helps you plan, not just a list of due dates.
2. **Who made it?** Team Nexora.
3. **Where can I try it?** https://deadline-tracker-team-nexora.vercel.app. Create an account; the first one on the server is the admin.
4. **Is it free?** Yes. It runs on free hosting, and Help uses Gemini's free tier with daily limits.
5. **Does it work on phones?** Yes, in the phone browser. The layout adapts; there's no installable app yet.
6. **Which users is it for?** Students first, then small teams and personal use, through three workspace types.
7. **What's the one thing to remember about it?** It answers "am I going to make it, and what should I do now?", and shows why.

B. Problem statement and objectives

8. **What problem does it solve?** Missed and last-minute deadlines caused by late starts, too many tasks and clashing due dates.
9. **Why isn't a reminder enough?** A reminder tells you a date is near; it doesn't tell you whether there's enough time left to do the work. The radar compares the work left with the hours you have.
10. **What's the main objective?** Finishing on time with less stress, while keeping personal data private.
11. **How does it reduce last-minute work?** Estimates + backward plans + the study planner spread work over earlier days, and the radar warns early.
12. **How does it help with prioritising?** Risk levels with reasons, and My Plan's "organise" orders tasks by urgency, explaining each one.
13. **Can it read my college portal automatically?** No. Tasks are entered by you (quickly, from one box). Importing timetables is a future idea.

C. Features and functionality

14. **How do I add a task?** Type it in the Add box, e.g. `Java assignment due Friday 5pm`, or press N for Quick add.
15. **What's the difference between a to-do and a deadline?** A deadline has a hard due date and time; a to-do is planned for a day. A task can have both.
16. **Can I add several tasks at once?** Yes: separate them with commas or new lines.
17. **How are priorities set?** Type `!high`, `urgent` or `!low`, or choose in the task panel. The assistant suggests high priority for exams and anything due within 2 days.

18. **What is the Deadline Radar?** A view where each deadline is placed by urgency and coloured On track, Needs attention, At risk, Critical or Overdue, with the reason.
19. **How is risk calculated?** Work left (estimate × (1 − progress), including earlier deadlines) compared with the hours you have before the due date, using your hours per day and work days. It's a clear rule, not AI.
20. **What if I don't give an estimate?** The radar uses only the date, and the app lists the task under "Needs an estimate".
21. **Is there a calendar?** Yes, month and week views; drag tasks to change their day.
22. **Can I search?** Yes, with the search box; the Deadlines page also has filters and grouping.
23. **Is there undo?** Yes: after deleting, moving or other changes, an Undo button appears for a few seconds.
24. **What is Rescue mode?** A four-step helper for late work: smaller steps, a revised schedule, clearing blockers, and a ready-made extension message you send yourself.
25. **What are progress check-ins?** Quick questions ("How far along is X?") that update progress so risk stays accurate.
26. **What is a focus session?** A 25-minute timer started from a task; the time logged improves estimate accuracy in Reports.
27. **Are reminders real?** Yes. With an account, the server creates them on time in your time zone even while the app is closed; you see them in the bell, and as desktop pop-ups while the app is open.
28. **Can I export to Google Calendar?** You can download a .ics file and import it. Live sync isn't available yet.
29. **Is there dark mode?** Yes, with the moon/sun button.
30. **Is there a Reader mode?** No. The team decided to skip it.

D. Uniqueness and competitors

31. **Why not just use Google Calendar?** It's great for *when* things happen, but it doesn't estimate whether your remaining work fits before a deadline or suggest a rescue plan. Many students use both: our .ics export helps.
32. **How is it different from Notion?** Notion is a flexible toolkit you set up yourself. Deadline Tracker is ready-made for deadlines: risk, backward planning and a day plan out of the box.
33. **How is it different from Todoist?** Todoist is excellent at fast task capture and organising. Our focus is on whether you'll finish in time (work left vs hours available) and what to do when you won't.
34. **Is anything here impossible for competitors?** No, and we don't claim that. Our difference is the focus, the explainable risk rules, Rescue mode and privacy-first AI, packaged for students.
35. **What's the USP in one line?** It warns you before you fall behind, explains why and plans the fix, while keeping your tasks out of AI training data.

E. User experience

36. **Is it hard to learn?** No: one Add box, a calm Today page and a four-item sidebar. Typing is like sending a message.
37. **Does it understand Hinglish?** Yes, common phrases, e.g. `kal Java assignment submit karna hai`, `aaj kya karu`.
38. **What if I make a spelling mistake?** Common ones ("tommorow", "delte a taks") are understood.

39. **Why does the site say "Starting up..."?** The free server sleeps after 15 minutes without visits and takes up to a minute to wake.
40. **Can I use it without an account?** On the live site, no; an account keeps your data safe. A copy opened without a server works in the browser only.
41. **Will my sample data mix with my real tasks?** You can clear sample data any time with **Clear**.

F. AI and Gemini

42. **What is Deadline AI?** The assistant, with two parts: My Plan (plans your tasks with our own rules) and Help (answers questions with Gemini).
43. **Does the AI see my tasks?** No. My Plan runs on our server without an outside AI; Gemini only gets Help questions and public help text.
44. **Does the AI change my tasks by itself?** No. My Plan only suggests; nothing is saved until you click Save, and the server checks every change.
45. **Which Gemini models do you use?** `gemini-3.5-flash-lite` for answers and `gemini-embedding-001` for search. Google listed both as free on the free tier when we set this up; the names can be changed in settings if Google retires them.
46. **What is RAG?** Retrieval-augmented generation: we find the matching parts of our help guide first, then let Gemini write the answer from them.
47. **How do you stop the AI making up features?** Website answers must come from guide sections; answers that claim website features without them are replaced with "I don't know", and the question is sent to the admin.
48. **Can Help answer general questions?** Yes, by default (coding, studies, general knowledge), and it says when it's unsure. An admin can switch to Focused mode.
49. **What happens when the free limit is reached?** Help answers website questions from the guide and says the AI limit was reached; My Plan keeps working; nothing paid is used.
50. **Is the AI trained on our data?** We don't train any model. Gemini is used as-is; our "training" is the help guide that the admin can update.

G. Backend and database

51. **What backend do you use?** Node.js with only built-in modules: no Express, no npm runtime packages.
52. **Which database?** SQLite, a real SQL database stored in one file, with migrations, constraints and indexes.
53. **How does the frontend talk to the backend?** JSON over HTTPS to `/api/...`. Vercel forwards those requests to the Render server.
54. **How are tasks stored?** In a `tasks` table with columns for title, dates, priority, status, estimate, progress and more; other app settings are in `app_state`.
55. **What if two devices edit at once?** Each save sends the version it last saw; if the server has a newer one, it refuses (409) and the app loads the newest copy instead of overwriting.
56. **How do reminders work when the site is closed?** A job inside the server runs every minute and stores due notifications; they're shown when you next open the app.
57. **Is there a public API?** Yes, a documented REST API for tasks, planning, notifications, statistics and admin (`docs/API.md`), all behind log-in.

H. Technology stack

58. **What is the frontend built with?** Plain HTML, CSS and JavaScript in one page, built from source parts by a small script. No framework.
59. **Why no React or Tailwind?** The app started as a single-file prototype; plain JavaScript keeps it fast and dependency-free. A framework could be adopted later if the team grows.
60. **Where is it hosted?** Frontend on Vercel, backend on Render, code on GitHub.
61. **How is it tested?** 269 automated tests: unit, API (against a real server with a fake Gemini) and browser tests in real Chromium on desktop and phone sizes.

I. Security and privacy

62. **How are passwords stored?** As scrypt hashes with a unique salt, never as text.
63. **Can one user see another's tasks?** No. Every query is limited to the logged-in user; this is tested.
64. **Where is the API key?** Only on the Render server. It's never in the browser, in GitHub or in responses.
65. **What does the admin see?** Accounts, help content, Help questions and usage, but not anyone's tasks.

J. Scalability and future

66. **Can it handle thousands of users?** Not on the free plan. The code is ready to move: a paid server with a disk, then PostgreSQL, a separate reminder worker and shared rate limits.
67. **What's next?** Email or Telegram reminders, shared team workspaces, calendar sync, an installable phone app.

K. Limitations

68. **What are the biggest current limits?** On the free Render plan, data is erased on redeploy or restart and the server sleeps; reminders are in-app only; team members aren't real accounts yet; tasks must be entered by hand.

L. Business and real-world use

69. **Could this become a product?** Possibly: free for students, with paid team features (shared workspaces, integrations) or college licences. We haven't validated this with paying users yet.
70. **Who would pay?** Most likely colleges, coaching centres or small teams that want workload visibility; students are the free core.

11. Difficult questions from judges and faculty (32)

Answer calmly, honestly and briefly. If something isn't built, say so, then say how you would build it.

1. **Why did you choose this problem?** *Every one of us has submitted something at the last minute. The issue isn't forgetting; it's late starts and clashing deadlines. A list can't warn about that, so we built something that checks whether the remaining work fits in the time left.*
2. **Why not just use Google Calendar?** *Calendar shows when things happen. It doesn't know how many hours your assignment needs, so it can't warn that you're already behind. We do that, and you can still export to Calendar with a .ics file.*
3. **How is this different from Notion?** *Notion is a blank toolkit you design yourself. We're ready-made for deadlines: risk with reasons, backward plans, a daily plan and Rescue mode, with no setup.*
4. **What is your USP?** *We tell you if you'll finish on time, why, and what to do next, while your tasks never go to an outside AI.*
5. **Why does a student need another productivity app?** *Most apps store tasks. Students need help deciding what to do today and noticing trouble early. That's our focus, and adding work is as fast as typing a message.*
6. **What if the user forgets to enter a task?** *Then we can't help with that task. No app can track what it doesn't know. We make entry very fast (one box, several tasks at once, Hinglish) to reduce that. Importing from college portals is a future step.*
7. **How do you handle conflicting deadlines?** *Clash detection flags overloaded weeks. The planner spreads work earliest-deadline-first within your hours per day and lists what won't fit. "Too much work" in My Plan shows the order and which days are overloaded.*
8. **Does it work without the internet?** *The hosted version needs the internet to reach the server. The same page can run without a server and save in the browser, but full offline sync isn't built. A service worker with sync is on the roadmap.*
9. **How do you protect private information?** *Hashed passwords (scrypt), HttpOnly session cookies, every query limited to the logged-in user, the AI key only on the server, and tasks never sent to Gemini. All of this is covered by automated tests.*
10. **How does your AI assistant work?** *Two parts. My Plan uses our own planning rules on our server to suggest tasks and day plans, which you approve. Help uses Gemini with RAG: we find the matching help-guide sections first, then Gemini writes the answer from them.*
11. **What happens when the Gemini API is unavailable?** *Help falls back to showing the matching help-guide section and says the AI is unavailable. My Plan doesn't depend on Gemini, so it keeps working.*
12. **Does AI automatically create or modify tasks?** *No. It only suggests. Saving requires a click, and the server re-checks every change and the task's owner.*
13. **How do you prevent incorrect AI information?** *For website questions, Gemini must answer from retrieved guide sections and return which ones it used; answers without them become "I don't know" and go to the admin. For general questions, it's told to say when it's unsure. We also show sources and let users rate answers.*
14. **How can you support thousands of users?** *Today it's on a free plan for demos. To scale: move SQLite to PostgreSQL, run the reminder job as one separate worker, put rate limits in*

Redis, then run several API servers behind a load balancer. The code is split into modules, so these are contained changes.

15. **What were your biggest technical challenges?** *Time zones for reminders (a 5 pm deadline must remind at 5 pm local time, including clock changes); never sending duplicate reminders; keeping the old browser-only app working while adding accounts; and stopping the AI from inventing features.*
16. **What are the current limitations?** *On the free Render plan, data is erased on redeploy and the server sleeps; reminders are in-app only (no email or SMS); team members aren't separate accounts yet; estimates depend on the user.*
17. **How will you maintain it?** *269 automated tests run before changes, numbered database migrations, documentation (architecture, API, deployment) and a team handbook. The code has no runtime dependencies to keep updated.*
18. **How will you attract students?** *A free tier, the college network (class groups, clubs) and a demo-first pitch: show the radar catching a real clash. Later, partnering with colleges for timetable import.*
19. **What's your future plan?** *Email or Telegram reminders, shared team workspaces with invites, calendar sync, an installable phone app, and smarter estimates from history.*
20. **What would you improve with more time?** *A persistent database host, email reminders and password reset, real team invites, and a password-change button in the UI.*
21. **Why plain JavaScript instead of React?** *It began as a single-file prototype that had to run anywhere, including inside a chat artifact. Plain JS kept it dependency-free and fast. If the team grows, migrating views to a framework is possible because rendering is already organised by view.*
22. **Why SQLite and not MySQL or MongoDB?** *It's a real SQL database with transactions and constraints, and it's built into Node 22.5+, so there's nothing to install and it's easy for a student team. For many users we'd move to PostgreSQL; the SQL is standard and in one place per module.*
23. **How do you know a reminder isn't sent twice?** *Every notification has a unique key per user (task + reminder time + deadline). The database enforces it with a UNIQUE constraint, and the tests run the job repeatedly to check.*
24. **What happens if the server was off when a reminder was due?** *When it comes back, only the most recent due reminder per task is delivered and earlier ones are marked skipped, so the user doesn't get a pile of messages.*
25. **How does the risk calculation work, exactly?** *For each deadline: hours of work left (estimate × (1 − progress), plus deadlines due earlier) divided by the hours available before it (your hours per day on work days). Above the thresholds (by default 40%, 70% and 100%) it becomes Needs attention, At risk, or Critical. Unfinished tasks it depends on raise it.*
26. **How do you stop one user reading another's data through the API?** *The user id always comes from the session cookie, never from the request. Every query includes* `user_id = ?`*. Another user's task id returns 404. There are tests for tasks, plans, conversations, notifications and ratings.*
27. **What's a vector database and do you use one?** *It stores embeddings and finds the closest ones. We store Gemini embeddings in SQLite and compare them in code (cosine similarity), which is enough for a help guide; a dedicated vector database would be overkill here.*
28. **Why combine keyword search with semantic search?** *Embeddings capture meaning; keywords catch exact terms, Hinglish words and typos we map ourselves. Combining the two rankings is more reliable, and keyword search still works if embeddings are unavailable.*

29. **How do you handle the Gemini free-tier limits?** *Our own limits sit below Google's: 8 per minute, 400 per day, 40 per user per day. After a 429 the server pauses for the time Google asks. Every call is logged for the admin. There's no paid fallback.*
30. **Isn't sending questions to a free AI a privacy risk?** *That's why only Help questions and public guide text are sent, never tasks. The Help window tells users that free-tier requests may be used by Google and asks them not to type private details.*
31. **How did you test the AI without paying?** *A fake Gemini server in our tests returns realistic answers, quota errors, empty and invalid responses, so we test every path for free. We don't claim tests replace trying it with the real key.*
32. **What if Google retires the model you use?** *The model names are settings (`GEMINI_MODEL`, `GEMINI_EMBED_MODEL`), so switching is a configuration change. The admin page shows errors if a model stops working.*

12. Demonstration scripts

Before you start: open the live site 2 minutes early (to wake the server), log in, and use a **Student** workspace with sample data. Clear notifications. Zoom the browser to 110–125%. Have a second tab with `/admin`.

12.1 Five-minute demo

Time	Do this	Say this
0:00–0:30 **Intro**	Show the Today page	"We're Team Nexora. This is Deadline Tracker, a task app that tells you whether you'll finish on time, not just what's due."
0:30–1:00 **Problem**	Point to "things need your attention"	"Students rarely forget deadlines entirely. We start too late, or three deadlines land in one week. A normal list only turns red when it's too late."
1:00–1:40 **Overview**	Show the sidebar, the Today list and Next deadlines	"Today shows what to do now: today's tasks, the next deadlines and anything risky. Four main pages, nothing more."
1:40–2:20 **Add a task**	Type `Java assignment due Friday 5pm ~4h, buy notes, call teammate tomorrow` → Add	"One box, like sending a message. It understood one deadline with a 4-hour estimate and two to-dos."
2:20–3:00 **Deadlines**	Open **Deadlines** → point at the radar → open an At risk item	"Each deadline gets a risk level with the reason: work left versus the hours I have. It's a clear rule, not a black box." Click **Suggest a backward plan**: "It splits the work into steps before the due date."
3:00–3:30 **Rescue**	Open an Overdue item → **Rescue**	"If I'm already late, Rescue mode gives smaller steps, a new schedule and an extension message I send myself. Nothing changes my deadline unless I confirm."
3:30–4:20 **AI**	Deadline AI → **My Plan**: `plan my tomorrow` → show reasons → **Save**. Then **Help**: `How do I delete a task?`	"My Plan plans from my own tasks on our server; my tasks never go to an outside AI. I approve before anything is saved. Help uses Google Gemini, but website answers come only from our help guide, with sources. If it doesn't know, it says so."
4:20–4:45 **Why different**	Show the bell / Reports	"Reminders come from the server in my time zone, even when the app is closed. Reports and statistics come from real data."
4:45–5:00 **Close**	Back to Today	"Deadline Tracker warns you before you fall behind, explains why and helps you fix it, privately. Thank you."

If the AI is slow or out of quota: "Help falls back to the guide, and My Plan doesn't need Gemini at all." Then show My Plan.

12.2 Two-minute demo

1. **(0:00)** "Deadline Tracker tells you if you'll finish on time." Show Today.
2. **(0:20)** Add `DBMS assignment due Fri 5pm ~5h, Maths test Monday` → "One box, two deadlines."
3. **(0:45)** Deadlines → radar → an At risk reason → "Work left versus hours available."
4. **(1:10)** Deadline AI → My Plan → `plan my tomorrow` → Save → "Suggestions from my own tasks, saved only when I approve. Private: no outside AI."
5. **(1:35)** Help → `How do I set a reminder?` → "Gemini answers from our help guide, with sources."
6. **(1:50)** "Warns early, explains why, plans the fix. Thank you."

12.3 Speaking tips

- Lead with the problem, then show; don't read features out.
- Use the words **"warns before"**, **"explains why"** and **"you approve"**.
- If something fails, stay calm, explain what should happen, and move on.

13. Team preparation

Project summary (memorise)

Deadline Tracker is a web app for students, teams and personal use that tells you whether you'll finish on time. A clear-rule risk radar compares work left with hours available. Backward plans, a study planner, clash warnings and Rescue mode help you act. Deadline AI has **My Plan** (private planning on our server, with your approval) and **Help** (Gemini with RAG on our help guide). Accounts keep data private; the server sends reminders in your time zone even while the app is closed. Built with plain JavaScript, Node.js and SQLite; hosted on Vercel and Render; 269 automated tests.

Important terms

Term	Meaning
Deadline vs to-do	A hard due date vs a planned day (a task can have both)
Estimate	Hours you think a task needs
Risk radar	Risk level per deadline, with the reason
Backward plan	Steps planned back from the due date
Rescue mode	A helper for late work (steps, schedule, extension message)
Workspace	Student, Company or Personal space
My Plan / Help	The two parts of Deadline AI
RAG	Find guide text first, then let the AI write the answer from it
Embedding	Numbers that represent the meaning of text
API	The server's addresses the page calls (`/api/...`)
Session cookie	Keeps you logged in; scripts can't read it
Free tier	Google's no-cost Gemini usage, with limits

Features every teammate must be able to show

Add box (several tasks at once, a deadline with an estimate) · Today tabs · Deadline Radar and its reasons · task panel (edit, estimate, backward plan) · Rescue mode · Calendar drag · Reminders settings and the bell · Deadline AI My Plan (`plan my tomorrow` , Save) and Help · Reports · theme · /admin.

Technical facts everyone should know

- Frontend: plain JS/HTML/CSS; backend: Node.js built-ins; database: SQLite.
- Vercel serves the pages and forwards `/api` to Render.
- Passwords are scrypt hashes; cookies are HttpOnly; every query is per user.

- Gemini models: `gemini-3.5-flash-lite` and `gemini-embedding-001`; our limits are 8/min, 400/day and 40/user/day.
- Tasks are never sent to Gemini.
- The reminder job runs every minute, deduplicated, time-zone aware.
- 269 automated tests.

Common mistakes to avoid

- Avoid: "Our AI plans everything" → [Implemented] "My Plan uses our own rules; Gemini only does Help."
- Avoid: "We send email or WhatsApp reminders" → [Implemented] "In-app bell and desktop alerts; email is planned."
- Avoid: "Teams can invite members" → [Implemented] "Team members are names in one workspace; invites are planned."
- Avoid: "No other app can do this" → [Implemented] "Our focus and approach are different."
- Avoid: "It's fully scalable" → [Implemented] "It runs on a free plan; here's the path to scale."
- Avoid: "Data is stored forever" → [Implemented] "On the free plan, data resets on redeploy; a paid disk fixes it."
- Avoid: Inventing numbers (users, accuracy). We have no user statistics yet.

Questions every teammate must answer

What is it? · What problem? · How is risk calculated? · How is it different from Google Calendar? · Does AI see my tasks? · What happens when Gemini fails? · How are passwords stored? · What are the limits? · What's next?

Quick revision sheet

- **Line:** tells you if you'll finish on time.
- **Three pillars:** warn early (radar), plan (backward plan, day plan, Rescue), remind (server job).
- **AI:** My Plan = private rules + approval; Help = Gemini + RAG + sources.
- **Stack:** JS · Node · SQLite · Vercel + Render · Gemini free tier.
- **Safety:** scrypt, HttpOnly cookie, per-user queries, key on the server only.
- **Limits:** free Render (resets, sleeps), in-app reminders only, no real team invites.

Final checklist (day before and one hour before)

- [ ] Everyone can give the one-minute intro.
- [ ] Each person has done the 2-minute demo once.
- [ ] The live site opens; you can log in; Deadline AI answers (check the Gemini key on Render).
- [ ] The account still exists (free Render resets data on redeploy; re-create it if needed).
- [ ] Sample data ready; notifications cleared.
- [ ] A backup plan: the app running locally (`node server.js`) if the internet fails.
- [ ] Everyone has read sections 11 and 13.

14. Future roadmap [Planned] (none of this is built yet)

Stage	Idea	Why it matters
Next	**Persistent database** (paid Render disk or managed PostgreSQL)	Data survives restarts
Next	**Email / Telegram reminders**, password reset email	Reminders reach users anywhere
Next	**Password change and profile page** in the UI	The server already supports password changes
Soon	**Shared team workspaces** with invites and server-side permissions	Real collaboration
Soon	**Calendar sync** (Google, Outlook) both ways	No double entry
Soon	**Installable app (PWA)** with offline mode	Phone-first students
Later	**Smarter estimates** learned from focus sessions and history	Better risk accuracy
Later	**Personalised suggestions** (best study times, workload balancing)	Proactive help
Later	**Timetable / portal import** with colleges	Less manual entry
Later	**Deeper analytics** (trends, semester reports)	Insight over time
Later	**Optional AI planning** with a paid, private AI tier (user opt-in)	Richer plans without free-tier privacy concerns

15. Glossary

API: the set of server addresses the app uses to read and save data. **Backend**: the server part (Node.js on Render). **Frontend**: what you see in the browser (on Vercel). **Database**: where data is stored (SQLite). **Migration**: a numbered change to the database structure. **Hash**: a one-way scramble of a password. **Cookie**: a small value the browser sends with each request to keep you logged in. **Rate limit**: a cap on how many requests are allowed in a time window. **RAG**: retrieval-augmented generation. **Embedding**: a list of numbers representing meaning. **Cosine similarity**: a measure of how close two embeddings are. **BM25**: a keyword-search scoring method. **Free tier**: limited no-cost use of a paid service. **429**: the HTTP code for "too many requests". **Transaction**: several database changes that succeed or fail together. **Time zone**: your local clock region (e.g. Asia/Kolkata).